Product Requirements Document (PRD)

Dragon Psycho QR Generator

Document version: 1.0
Product name: Dragon Psycho QR Generator
Brand identity: DRAGONQR
Product type: Free, open-source QR code generation web application
Development platform: Google AI Studio
Frontend: React + TypeScript + Tailwind CSS
Backend and database: Supabase
Authentication: Not required
Primary design: Minimal black, white and grey, based on the provided reference screenshot.

---

1. Product Overview

Dragon Psycho QR Generator is a free web application that allows users to generate, customize, preview, download and manage QR codes without creating an account.

The application will support both static and dynamic QR codes. Static QR codes encode their content directly, while dynamic QR codes use a redirect URL whose destination can be updated after creation.

The interface must follow the user's provided reference screenshot, including the DRAGONQR branding, professional typography, QR configuration panel, live output preview and clean black-and-white design.

The website will have four main navigation sections:

1. Home
2. Donate
3. History
4. Dragon Psycho

The application must not collect QR scan analytics or track scanning devices.

2. Product Goals

Primary goals

- Provide a professional, easy-to-use QR code generator.
- Support multiple QR content types.
- Provide static and dynamic QR code generation.
- Allow detailed visual customization.
- Support PNG and SVG downloads.
- Store generated QR history locally in the user's browser.

- Store dynamic QR destinations in Supabase.
- Provide a donation page linked to Cashfree.
- Include privacy, terms, contact and social media pages.
- Keep the core QR generation features free to use.

Non-goals

The initial release will not include:

- User registration or login.
- Paid subscriptions or premium feature restrictions.
- QR scan counts, device tracking or scan analytics.
- vCard/contact-card QR generation.
- Cloud synchronization of browser history across devices.
- An administrator dashboard unless required in a later release.

3. Target Users

- Individuals creating personal QR codes.
- Small businesses and shop owners.
- Content creators and social media users.
- Users sharing Wi-Fi details, links or contact numbers.
- Users who need editable destination URLs through dynamic QR codes.

4. Information Architecture

4.1 Home

The Home page is the primary QR generation workspace.

It must include:

- DRAGONQR logo and branding.
- QR content-type selection.
- Static/dynamic QR mode selector.
- Input fields relevant to the selected QR type.
- QR styling and customization controls.
- Live QR preview.
- PNG and SVG download controls.
- Clear/reset controls.
- Input validation and readable error messages.

4.2 Donate

The Donate page will explain that users can voluntarily support the project.

Requirements:

- Display a short donation message.
- Include a clearly visible Donate button.
- Open the configured Cashfree payment link when clicked.
- Open the payment page securely, preferably in the same tab or through a standard external-link flow.
- Display a helpful message if the payment link has not been configured.

The actual Cashfree payment URL will be supplied by the product owner before launch. Do not invent a payment link or payment credentials.

4.3 History

The History page will display QR codes generated in the current browser.

Each saved entry should include:

- QR label or name.
- QR content type.
- Static or dynamic mode.
- Creation timestamp.
- QR preview, where available.
- Preview or reopen action.
- Download action.
- Delete action.

History should support clearing all entries after confirmation.

History data will be stored using browser local storage or IndexedDB, with IndexedDB preferred if larger records or uploaded logo data need to be retained.

Important limitations:

- History is specific to the browser and device.
- Clearing browser storage can erase history.
- History will not automatically synchronize across devices.
- Dynamic destination records in Supabase and browser history are separate data stores.

4.4 Dragon Psycho

This section will provide project information and navigation to supporting pages.

Required links:

1. Privacy Policy
2. Terms and Conditions
3. About Dragon Psycho
4. Contact
5. Social Media

Privacy Policy and Terms and Conditions must open dedicated pages rather than simply displaying unstructured text.

Contact details can remain configurable placeholders until the owner provides them.

The Social Media section should appear near the bottom in a visually distinct black square-style panel containing social platform icons and links.

5. Global Navigation

A persistent bottom navigation bar is required.

Navigation items:

Item| Icon suggestion| Action
Home| Home| Open QR generator
Donate| Heart/handshake| Open Donate page
History| History| Open saved QR entries
Dragon Psycho| User/profile| Open project information

Requirements:

- Highlight the currently active section.
- Use a responsive layout for mobile and desktop.
- Keep the bar readable and accessible.
- Avoid obscuring the main content on smaller screens.
- Support keyboard navigation on desktop.
- Use consistent Lucide icons.

A top header should display the DRAGONQR logo and an optional menu for secondary links.

6. QR Code Generation Requirements

6.1 Supported QR content types

The following types must be available:

Type| Required input
URL / Link| Destination URL
Text| Plain text
Wi-Fi| Network name, security type and password where applicable
Email| Email address, optional subject and body
Phone| Phone number
SMS| Phone number and optional message
WhatsApp| Phone number and optional prefilled message
Location| Latitude/longitude or a supported map URL

Event| Event name, start/end time, location and optional description
Social| Social platform and profile URL

vCard must not appear anywhere in the QR type selector.

Phone number generation must remain available as its own option.

6.2 QR generation flow

1. User selects a content type.
2. The application displays the relevant input fields.
3. The user enters content and an optional label.
4. The user chooses static or dynamic mode where supported.
5. The user customizes the QR appearance.
6. The application validates the input.
7. The user selects Generate QR Code.
8. A live QR preview appears.
9. The generated entry is saved in browser history.
10. The user can download the QR code.

Do not generate a new database record on every preview render. Only create or update records when the user explicitly saves or generates a dynamic QR code.

6.3 Static QR codes

Static QR codes encode their content directly.

Requirements:

- Generate without requiring a backend connection.
- Work for all supported content types where technically appropriate.
- Support PNG and SVG export.
- Allow styling customization.
- Remain functional if the website's database is unavailable.

Static QR content cannot be remotely edited after the QR code is downloaded.

6.4 Dynamic QR codes

Dynamic QR codes must use a stable redirect URL managed by the application.

Example format:

"https://YOUR-DOMAIN.COM/r/UNIQUE_ID"

The final domain must be configurable. The example above is a format illustration, not a live URL.

Requirements:

- Create a unique identifier for each dynamic QR code.
- Save the identifier and destination URL in Supabase.
- Generate the QR code using the stable redirect URL.
- Allow the owner of the QR entry to update the destination.
- Keep the QR code's redirect identifier unchanged when the destination is edited.
- Allow deletion or deactivation of dynamic QR entries.
- Show an appropriate unavailable/expired message for inactive entries.
- Validate destination URLs before saving.
- Provide a public redirect endpoint that resolves the identifier to the current destination.

No scan tracking: The redirect process must not store scan counts, IP addresses, device identifiers, user-agent details or scan history. Avoid analytics middleware or request logging that captures these details for the QR redirect endpoint. Essential security and infrastructure logs must be configured with privacy minimization and retention limits.

6.5 Dynamic QR without user accounts

Because login is not required, the application must not expose unrestricted destination editing or deletion to anyone who knows a QR identifier.

Use a secure ownership mechanism, such as a randomly generated, high-entropy management token, with a server-side hash stored in Supabase. The original token should be shown to the creator only when the dynamic QR is created and should not be embedded in the public QR URL.

Management operations must be verified server-side. Never expose the Supabase service-role or secret key in frontend code.

Provide clear recovery limitations: if the management token is lost, the user may be unable to edit that dynamic QR code.

7. QR Styling and Customization

The Home page must include a professional styling panel inspired by the screenshot.

Required controls

Colors

- QR foreground color.
- Background/canvas color.
- Color contrast validation.

Module patterns

- Support multiple QR module patterns where the selected QR library allows them.

- Include a smooth/squircle-style option if supported.

Corner styling

- Corner eye/frame style.
- Corner eyeball style.

Spacing

- Quiet-zone/margin selection.
- Preserve an adequate margin for reliable scanning.

Output

- Square canvas by default.
- Configurable export resolution.
- Error-correction level selection: L, M, Q and H where supported.
- PNG download.
- SVG download.

Logo

- Upload a logo image.
- Preview the logo inside the QR code.
- Adjust logo size.
- Remove or replace the logo.
- Validate file type and size.
- Warn users when a logo may reduce scan reliability.

The application must prioritize scan reliability over visual decoration. Provide sensible defaults and test generated outputs with multiple scanning applications.

8. Live Output Preview

The preview panel must update when the user changes QR content or supported styling options.

It should display:

- Current QR code.
- Generation/readiness status.
- Selected foreground and background colors.
- Export actions.
- Validation warnings where relevant.

Preview rendering should be efficient and should not create unnecessary database writes.

If the QR cannot be generated, display a clear error rather than a blank preview.

9. History and Local Persistence

The application must save each successful generation to browser storage.

Recommended history record:

- Local record ID.
- Display label.
- Content type.
- Static/dynamic mode.
- Creation timestamp.
- QR content or a reference to the saved content.
- Styling settings.
- Dynamic QR identifier, if applicable.

Sensitive Wi-Fi passwords and other private content must not be sent to Supabase merely to support local history. Users should be informed that locally stored history is accessible to anyone who can use the same browser profile.

History operations:

- View entries.
- Reopen a saved QR.
- Download an entry.
- Delete a single entry.
- Clear all history after confirmation.

If storage is unavailable or full, show an understandable warning and preserve the current preview where possible.

10. Supabase Backend Requirements

Supabase will provide the database and server-side infrastructure required for dynamic QR codes.

10.1 Suggested data model

Table: "dynamic_qr_codes"

Column| Type| Purpose
"id"| UUID| Internal record identifier
"slug"| Text| Unique public redirect identifier
"destination_url"| Text| Current redirect destination
"label"| Text| Optional user-defined label
"is_active"| Boolean| Enable or disable redirect
"management_token_hash"| Text| Secure verification for management
"created_at"| Timestamp| Creation time

"updated_at"| Timestamp| Last update time

No scan-count, device, IP-address or scan-history columns should be created.

10.2 Database security

- Enable Row Level Security (RLS) on exposed tables.
- Deny unrestricted public reads and writes.
- Do not allow anonymous users to enumerate all dynamic QR records.
- Use server-side validation for creation, update and deletion.
- Keep the Supabase secret/service-role key on the server only.
- Use a publishable key in the frontend only where appropriate.
- Rate-limit dynamic QR creation and management endpoints.
- Validate redirect destinations and reject unsafe schemes.
- Avoid open-redirect vulnerabilities beyond the intended QR redirect behavior.
- Provide clear error handling for missing, disabled or invalid records.

10.3 Backend endpoints

The implementation should provide equivalent secure server-side functions for:

- Create dynamic QR.
- Resolve public QR redirect.
- Update destination.
- Deactivate/delete a dynamic QR.
- Validate management credentials.

These may be implemented using Supabase Edge Functions or another suitable server-side endpoint. The redirect endpoint must return the appropriate redirect response without recording scan analytics.

10.4 Availability

Static QR generation must remain available if Supabase is unavailable. Dynamic QR creation and destination editing may temporarily fail when the backend is unavailable, but the application must display a clear message and avoid silently losing user input.

11. Visual Design System

11.1 Design direction

Follow the provided screenshot closely.

The design should feel professional, minimal and technical, with a strong contrast between the content panel and the surrounding interface.

11.2 Color palette

Element| Suggested color
Primary dark| "#171717"
Main text| "#222222"
Secondary text| "#666666"
Page background| "#F5F5F5"
Card background| "#FFFFFF"
Borders| "#E3E3E3"
Active controls| "#171717"
Active control text| "#FFFFFF"

These are implementation starting points; the screenshot remains the visual reference.

Do not use purple or blue gradients.

11.3 Typography

Use a clean, professional sans-serif font for controls and body text. Use a restrained serif-like heading style only if needed to match the screenshot.

Maintain:

- Clear text hierarchy.
- Consistent spacing.
- Readable labels and helper text.
- Accessible contrast.
- Consistent button heights and corner radii.

11.4 Responsive layout

Desktop:

- Main content area with configuration controls and live preview.
- Two-column layout where appropriate.
- Styling controls grouped into clear panels.

Mobile:

- Single-column layout.
- Touch-friendly controls.
- Readable input fields.
- QR preview sized to the available viewport.
- Persistent bottom navigation with sufficient bottom padding.

12. Social Media Integration

The Social Media panel will use the following owner-provided links.

Platform| Destination

YouTube| https://www.youtube.com/@Dragon_music3e
Instagram|
https://www.instagram.com/dragon__prince_458?xtok=MXNuaDZkMXNxZG5nMg==
Facebook| https://www.facebook.com/share/1CLjxKeFFZ/
GitHub| https://github.com/pk9607959-maker

Requirements:

- Use recognizable platform icons.
- Make each icon and its associated label clickable.
- Open external links safely.
- Preserve the supplied destination URLs.
- Keep links easy to replace in a configuration file.

The DRAGONQR logo and text will appear in the primary header. Dragon Psycho will identify the project-information section.

13. Donation Integration

Cashfree will be used for donations through a payment link supplied by the owner.

Requirements:

- Keep the payment link configurable.
- Do not store payment secrets in frontend code.
- Do not claim that a payment succeeded merely because the user clicked Donate.
- Display only verified payment status if payment verification is added later.
- Keep donation optional; all QR features remain available without donating.

14. Privacy and Legal Requirements

Create dedicated Privacy Policy and Terms and Conditions pages.

The Privacy Policy must explain:

- What information is stored locally.
- What information is stored in Supabase for dynamic QR functionality.
- The absence of QR scan analytics and device tracking.
- Relevant third-party services.
- Data retention and deletion behavior.
- The limitations of browser-based history.

The Terms and Conditions must cover:

- Acceptable use.
- Responsibility for the content encoded in QR codes.
- Dynamic QR availability and destination changes.
- Service availability and limitations.

- Appropriate liability and warranty language.

The content must be reviewed before production launch and must not make unsupported legal or privacy guarantees.

15. Accessibility and Usability

- All controls must have accessible labels.
- Keyboard navigation must work.
- Focus indicators must be visible.
- Errors must be expressed in text, not color alone.
- Color contrast must be checked.
- Interactive elements must be touch-friendly.
- Forms must provide clear validation.
- Uploaded logos must have size and type restrictions.
- Confirm destructive actions such as deleting all history.

16. Technical Stack

Recommended implementation:

- React.
- TypeScript.
- Vite.
- Tailwind CSS.
- Lucide icons.
- A maintained QR generation library that supports the required formats and styling.
- Supabase JavaScript client.
- Supabase PostgreSQL database.
- Supabase Edge Functions or equivalent secure server-side functions.
- Browser local storage or IndexedDB for local history.
- Environment variables for configuration.

The implementation must verify that the chosen QR library supports the selected module patterns, corner shapes, logo embedding, SVG export and error-correction controls. Where a feature is unsupported, use a compatible library rather than displaying a nonfunctional control.

17. Configuration Requirements

Provide environment configuration for:

- Supabase project URL.
- Supabase publishable key.
- Cashfree payment link.
- Public website domain.
- Social media links.
- Optional support/contact information.

No secret keys should be committed to the public repository or bundled into client-side assets.

18. Acceptance Criteria

The first release is accepted when all the following are true:

1. The application loads and works on mobile and desktop.
2. The design follows the provided black-and-white screenshot.
3. The DRAGONQR logo is displayed in the header.
4. The bottom navigation contains Home, Donate, History and Dragon Psycho.
5. All ten supported QR content types are available.
6. vCard is absent from the interface.
7. Phone number QR generation works.
8. Static QR generation works without a backend.
9. Dynamic QR destinations are stored securely in Supabase.
10. Updating a dynamic destination does not change its QR redirect identifier.
11. Dynamic QR codes redirect to the current valid destination.
12. Scan analytics and device tracking are not implemented.
13. QR styling controls work and update the preview.
14. PNG and SVG downloads produce valid files.
15. Logo upload and logo sizing work.
16. Generated entries appear in local History.
17. Users can reopen, download and delete history entries.
18. Clearing history requires confirmation.
19. Donate opens the configured Cashfree payment link.
20. Privacy Policy and Terms and Conditions open separate pages.
21. All four supplied social links work.
22. No login or signup is required.
23. Backend secrets are not exposed in the frontend.
24. Database security rules prevent unrestricted record access.
25. Errors and unavailable-backend states are handled clearly.

19. Testing Plan
Functional tests
Generate a QR code for each supported content type.
Verify static QR output using multiple scanner apps.
Create a dynamic QR and confirm that it redirects correctly.
Change the destination and verify the original QR now opens the updated destination.
Confirm that no scan history or device analytics are stored.
Test PNG and SVG downloads.
Test logo upload, replacement and removal.
Test History persistence after reloading the page.
Test deleting individual and all history entries.
Verify the donation and social links.

Security tests
Confirm the Supabase secret key is not present in client assets.
Test unauthorized dynamic QR edits and deletions.
Test invalid or malformed redirect identifiers.
Test URL validation and unsafe redirect schemes.
Confirm users cannot enumerate all dynamic QR records.
Confirm rate limits and RLS policies work as intended.

Responsive tests
Test narrow Android screens.
Test common desktop resolutions.
Test navigation, preview and download actions on mobile.
Confirm the bottom navigation does not cover form controls.
20. Deployment Plan
Build and test the React application in Google AI Studio.
Export or connect the source code to a GitHub repository.
Create a Supabase project.
Apply the database schema and security policies.
Deploy the server-side dynamic QR functions.
Configure environment variables.
Configure the public website domain and redirect base URL.
Add the Cashfree payment link.
Test static and dynamic QR functionality.

Test the application on mobile and desktop.
Deploy the frontend to a compatible hosting provider such as Netlify or Vercel.
Verify the production domain, redirects, database security and download functionality.
The application should be open source, but hosting, database usage and domain registration may have costs or free-tier limits. Open-source status does not guarantee that every infrastructure service is free indefinitely.
21. Future Enhancements
Potential future improvements, subject to owner approval:
PWA installation.
Bulk QR generation.
Import/export of local history.
Multiple QR templates.
Additional export formats.
Improved offline static QR generation.
Optional encrypted backup and restore.
Administrative tools for managing public information pages.
These enhancements are not part of the initial release unless separately approved.
22. Final Product Definition
Dragon Psycho QR Generator is a responsive, free QR generation
platform with the DRAGONQR visual identity, static and dynamic QR support, detailed styling, PNG/SVG exports, local history, a Supabase-powered dynamic redirect system, Cashfree donations and a project-information section.

The application must not require login, must not implement QR scan analytics, must exclude vCard, and must retain the Phone QR option.
The provided screenshot is the primary visual design reference for the implementation.